Name : Manan Dudeja

Roll No: 24BCE1270

EXPERIMENT - 10

AIM : To implement memory allocation algorithms (First Fit, Best Fit, and Worst Fit) and allocate memory blocks to processes using variable partitioning.

ALGORITHM:

First Fit Algorithm

1. Start scanning memory blocks from the beginning.
2. Select the first block that is large enough for the process.
3. Allocate the process to that block.
4. Reduce the block size by the allocated amount.
5. Repeat for all processes until allocation finishes.

Best Fit Algorithm

1. Check all available memory blocks.
2. Choose the smallest block that is large enough for the process.
3. Allocate the process to that block.
4. Reduce the remaining block size.
5. Repeat for all processes.

Worst Fit Algorithm

1. Check all memory blocks.
2. Choose the largest available block for the process.
3. Allocate the process.
4. Reduce the remaining block size.

5. Repeat until all processes are allocated or memory is insufficient.

PROGRAM:

```
#include <iostream> using
namespace std;

void firstFit(int blockSize[], int m, int processSize[], int n) {     int
allocation[n];
   for(int i=0;i<n;i++) allocation[i] = -1;

   for(int i=0;i<n;i++){
for(int j=0;j<m;j++){
          if(blockSize[j] >= processSize[i]){
allocation[i] = j;
              blockSize[j] -= processSize[i];
break;
          }
       }
   }

   cout << "\nFirst Fit Allocation\n";     cout
<< "Process\tSize\tBlock\n";     for(int
i=0;i<n;i++){
       cout << i+1 << "\t" << processSize[i] << "\t";
if(allocation[i] != -1)            cout << allocation[i] + 1;
else
          cout << "Not Allocated";
       cout << endl;
```

```
    }
}

void bestFit(int blockSize[], int m, int processSize[], int n){     int
allocation[n];
    for(int i=0;i<n;i++) allocation[i] = -1;

    for(int i=0;i<n;i++){        int bestIdx = -1;
for(int j=0;j<m;j++){            if(blockSize[j] >=
processSize[i]){
                if(bestIdx == -1 || blockSize[bestIdx] > blockSize[j])
bestIdx = j;
            }
        }

        if(bestIdx != -1){            allocation[i] =
bestIdx;            blockSize[bestIdx] -=
processSize[i];
        }
    }

    cout << "\nBest Fit Allocation\n";     cout
<< "Process\tSize\tBlock\n";     for(int
i=0;i<n;i++){
        cout << i+1 << "\t" << processSize[i] << "\t";
if(allocation[i] != -1)            cout << allocation[i] + 1;
else
            cout << "Not Allocated";
cout << endl;
    }
```

```
}

void worstFit(int blockSize[], int m, int processSize[], int n){     int
allocation[n];
   for(int i=0;i<n;i++) allocation[i] = -1;

   for(int i=0;i<n;i++){        int worstIdx = -1;
for(int j=0;j<m;j++){            if(blockSize[j] >=
processSize[i]){
              if(worstIdx == -1 || blockSize[worstIdx] < blockSize[j])
worstIdx = j;
          }
       }

       if(worstIdx != -1){            allocation[i] =
worstIdx;            blockSize[worstIdx] -=
processSize[i];
       }
   }

   cout << "\nWorst Fit Allocation\n";     cout
<< "Process\tSize\tBlock\n";     for(int
i=0;i<n;i++){
       cout << i+1 << "\t" << processSize[i] << "\t";
if(allocation[i] != -1)            cout << allocation[i] + 1;
       else
          cout << "Not Allocated";
cout << endl;
   }
}
```

```
int main(){

   int blockSize[] = {100,500,200,300,600};     int
processSize[] = {212,417,112,426,95};

   int m = sizeof(blockSize)/sizeof(blockSize[0]);     int n =
sizeof(processSize)/sizeof(processSize[0]);

   int block1[m], block2[m], block3[m];

   for(int i=0;i<m;i++){
block1[i] = blockSize[i];
block2[i] = blockSize[i];
block3[i] = blockSize[i];
   }

   firstFit(block1,m,processSize,n);
bestFit(block2,m,processSize,n);
worstFit(block3,m,processSize,n);

   return 0;
}
```

```
#include <iostream>
using namespace std;

void firstFit(int blockSize[], int m, int processSize[], int n) {
    int allocation[n];
    for(int i=0;i<n;i++) allocation[i] = -1;

    for(int i=0;i<n;i++){
        for(int j=0;j<m;j++){
            if(blockSize[j] >= processSize[i]){
                allocation[i] = j;
                blockSize[j] -= processSize[i];
                break;
            }
        }
    }

    cout << "\nFirst Fit Allocation\n";
    cout << "Process\tSize\tBlock\n";
    for(int i=0;i<n;i++){
        cout << i+1 << "\t" << processSize[i] << "\t";
        if(allocation[i] != -1)
            cout << allocation[i] + 1;
        else
            cout << "Not Allocated";
        cout << endl;
    }
}

void bestFit(int blockSize[], int m, int processSize[], int n){
    int allocation[n];
    for(int i=0;i<n;i++) allocation[i] = -1;

    for(int i=0;i<n;i++){
        int bestIdx = -1;
        for(int j=0;j<m;j++){
            if(blockSize[j] >= processSize[i]){
                if(bestIdx == -1 || blockSize[bestIdx] > blockSize[j])
                    bestIdx = j;
            }
        }

        if(bestIdx != -1){
            allocation[i] = bestIdx;
            blockSize[bestIdx] -= processSize[i];
        }
    }

    cout << "\nBest Fit Allocation\n";
    cout << "Process\tSize\tBlock\n";
    for(int i=0;i<n;i++){
        cout << i+1 << "\t" << processSize[i] << "\t";
        if(allocation[i] != -1)
            cout << allocation[i] + 1;
        else
            cout << "Not Allocated";
```

```
        cout << endl;
    }
}

void worstFit(int blockSize[], int m, int processSize[], int n){
    int allocation[n];
    for(int i=0;i<n;i++) allocation[i] = -1;

    for(int i=0;i<n;i++){
        int worstIdx = -1;
        for(int j=0;j<m;j++){
            if(blockSize[j] >= processSize[i]){
                if(worstIdx == -1 || blockSize[worstIdx] < blockSize[j])
                    worstIdx = j;
            }
        }

        if(worstIdx != -1){
            allocation[i] = worstIdx;
            blockSize[worstIdx] -= processSize[i];
        }
    }

    cout << "\nWorst Fit Allocation\n";
    cout << "Process\tSize\tBlock\n";
    for(int i=0;i<n;i++){
        cout << i+1 << "\t" << processSize[i] << "\t";
        if(allocation[i] != -1)
            cout << allocation[i] + 1;
        else
            cout << "Not Allocated";
        cout << endl;
    }
}

int main(){

    int blockSize[] = {100,500,200,300,600};
    int processSize[] = {212,417,112,426,95};

    int m = sizeof(blockSize)/sizeof(blockSize[0]);
    int n = sizeof(processSize)/sizeof(processSize[0]);

    int block1[m], block2[m], block3[m];

    for(int i=0;i<m;i++){
        block1[i] = blockSize[i];
        block2[i] = blockSize[i];
        block3[i] = blockSize[i];
    }

    firstFit(block1,m,processSize,n);
    bestFit(block2,m,processSize,n);
    worstFit(block3,m,processSize,n);

    return 0;
```

OUTPUT:

```
(base) student@student:~$ vi hi.cpp
(base) student@student:~$ g++ hi.cpp
(base) student@student:~$ ./a.out

First Fit Allocation
Process Size    Block
1       212     2
2       417     5
3       112     2
4       426     Not Allocated
5       95      1

Best Fit Allocation
Process Size    Block
1       212     4
2       417     2
3       112     3
4       426     5
5       95      1

Worst Fit Allocation
Process Size    Block
1       212     5
2       417     2
3       112     5
4       426     Not Allocated
5       95      4
```

RESULT: The program successfully demonstrates memory allocation using First Fit, Best Fit, and Worst Fit algorithms.
The experiment shows how different strategies affect memory utilization and fragmentation.

Thus, the experiment demonstrates proper process synchronization in operating systems.